

Aula 08

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) SOA - Teoria	3
2) SOA - Questões Comentadas - FGV	27
3) SOA - Lista de Questões - FGV	38

ARQUITETURA ORIENTADA A SERVIÇOS (SOA)

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTÍSSIMA

ARQUITETURA ORIENTADA A SERVIÇOS (SOA)

A Arquitetura Orientada a Serviços (SOA) é um estilo arquitetural que organiza sistemas de software como um conjunto de serviços independentes, reutilizáveis e interoperáveis. Cada serviço encapsula uma funcionalidade de negócio específica e pode ser consumido por diferentes aplicações por meio de contratos padronizados. O principal objetivo da SOA é promover integração, reutilização, flexibilidade e alinhamento entre a tecnologia da informação e os processos de negócio da organização.

A Arquitetura Orientada a Serviços (SOA) baseia-se na organização de sistemas por meio de serviços independentes que disponibilizam funcionalidades através de interfaces bem definidas. Nessa abordagem, os consumidores não precisam conhecer a implementação interna dos provedores, acessando apenas os recursos expostos. Essa separação favorece a abstração da complexidade, promove a interoperabilidade entre aplicações e permite que funcionalidades sejam disponibilizadas de forma padronizada para diferentes consumidores.

A comunicação entre consumidores e provedores ocorre por meio de contratos de serviço. Esses contratos definem operações disponíveis, formatos de mensagens, estruturas de dados e regras de interação. Funcionam como uma linguagem comum capaz de garantir que sistemas distintos consigam se comunicar de maneira previsível. Dessa forma, os contratos estabelecem as bases para a integração entre aplicações, independentemente das tecnologias, plataformas ou linguagens utilizadas em sua implementação.

A existência de contratos conduz diretamente ao princípio do baixo acoplamento, um dos pilares da orientação a serviços. O objetivo não é eliminar completamente as dependências, mas reduzir a dependência dos consumidores em relação à lógica interna, à tecnologia e aos detalhes de implementação dos serviços. Assim, os consumidores passam a depender principalmente das definições contratuais, permitindo que alterações internas sejam realizadas com menor impacto sobre os sistemas que utilizam o serviço.

Embora consumidores e provedores dependam dos contratos para se comunicar, essa dependência não caracteriza necessariamente um acoplamento forte. Na literatura de SOA, os contratos são vistos como mecanismos de estabilização das integrações. Alterações incompatíveis nos contratos podem afetar consumidores existentes, razão pela qual mudanças costumam exigir versionamento e planejamento cuidadoso. Ainda assim, a dependência contratual é considerada mais saudável e controlada do que a dependência direta da implementação.

A utilização de contratos padronizados também viabiliza a interoperabilidade entre sistemas heterogêneos. Aplicações desenvolvidas em plataformas distintas, pertencentes a diferentes organizações ou executadas em ambientes diversos conseguem interagir desde que respeitem as especificações contratuais acordadas. Dessa forma, o contrato atua como um ponto de alinhamento que permite a comunicação entre partes independentes sem exigir conhecimento sobre processos internos ou tecnologias específicas.

Outro benefício importante proporcionado pela abstração e pelo baixo acoplamento é a reutilização de serviços. Como os consumidores dependem apenas das interfaces expostas, um mesmo serviço pode atender diferentes aplicações, departamentos ou processos de negócio. Nesse contexto surge o conceito

de serviço agnóstico, proposto por Thomas Erl. Um serviço é considerado agnóstico quando sua lógica não está vinculada a um contexto específico, ampliando significativamente seu potencial de reutilização.

Esses conceitos estão presentes na definição de SOA proposta pela OASIS, que descreve a arquitetura como um paradigma voltado à organização e utilização de recursos distribuídos sob diferentes domínios de controle. Além do baixo acoplamento e da reutilização, autores como Thomas Erl destacam características como contratos formais, abstração, autonomia e composição de serviços. Em conjunto, esses atributos aumentam a flexibilidade, a interoperabilidade e a capacidade de evolução dos sistemas corporativos. Abaixo segue uma lista de algumas definições de SOA que eu já encontrei em provas...

O QUE SOA É...

Um conjunto de princípios e melhores práticas para implementação e execução de processos de negócio automatizados em ambientes de tecnologia da informação heterogêneos.

Uma forma de aproximar a linguagem do negócio e da tecnologia da informação, facilitando a integração de ambientes corporativos por meio de serviços.

Um meio para organizar as soluções que promove o reúso, o crescimento e a interoperabilidade.

Uma abordagem distribuída (não-monolítica) para integração de arquiteturas baseadas no conceito de serviço.

Uma abordagem arquitetural corporativa que permite a criação de serviços de negócio interoperáveis, que podem ser reutilizados e compartilhados entre aplicações e empresas.



O surgimento da Arquitetura Orientada a Serviços está diretamente relacionado aos desafios enfrentados pelas organizações após anos de crescimento tecnológico. Para atender demandas específicas de negócio, inúmeras aplicações foram desenvolvidas de forma independente, sem uma estratégia integrada de arquitetura. Como consequência, os ambientes corporativos passaram a acumular sistemas isolados que precisavam compartilhar informações, tornando a integração cada vez mais necessária para viabilizar processos e operações do dia a dia.

Uma solução frequentemente adotada consistia em criar aplicações responsáveis por acessar diversos sistemas simultaneamente e consolidar informações para os usuários. Com o tempo, novas demandas

exigiam automatizações, relatórios periódicos e integrações adicionais. Embora cada solução resolvesse um problema específico, a criação contínua dessas conexões aumentava gradualmente a quantidade de dependências existentes, elevando a complexidade operacional e dificultando a gestão do ambiente tecnológico.

Quando esse padrão se repete por muitos anos, forma-se o cenário conhecido como Hairball Architecture, ou Arquitetura de Bola de Pelo. Nesse contexto, inúmeros sistemas encontram-se conectados por integrações ponto a ponto construídas para necessidades específicas. O resultado é uma rede complexa de relacionamentos, na qual identificar dependências, compreender impactos e promover alterações torna-se uma tarefa difícil, demorada e frequentemente arriscada para as organizações.

Os principais problemas observados nesse tipo de ambiente são o alto acoplamento e a redundância de funcionalidades. O alto acoplamento faz com que mudanças em uma aplicação afetem diversas outras, aumentando riscos e custos de manutenção. Já a redundância ocorre quando funcionalidades semelhantes são implementadas repetidamente em sistemas diferentes. Esses fatores reduzem a flexibilidade tecnológica e dificultam a evolução dos sistemas ao longo do tempo, comprometendo a eficiência da área de TI.

Foi para enfrentar esses desafios que a SOA ganhou importância. Seu objetivo é promover reutilização, reduzir dependências excessivas e facilitar a integração entre sistemas. Entretanto, SOA não é um produto nem uma tecnologia específica, mas um paradigma arquitetural baseado em princípios e padrões. Tecnologias como Web Services, CORBA, RPC, DCOM, DDS e WCF podem ser utilizadas para implementá-la, mas representam apenas mecanismos de implementação, e não a própria arquitetura orientada a serviços. Agora vejamos o que o SOA não é...

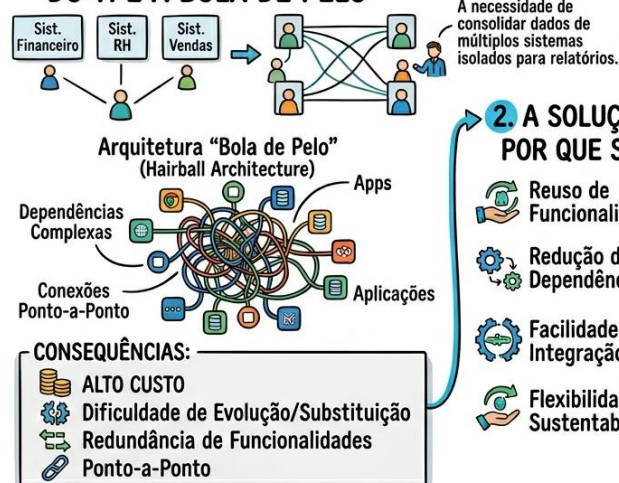
O QUE SOA NÃO É...

Não é uma tecnologia.	Não é um produto.	Não é um web service.
Não é um projeto de TI.	Não é um software.	Não é um framework.
Não é uma metodologia.	Não é solução de negócio.	Não é um middleware.
Não é um serviço.	Não é uma ferramenta.	

DESVENDANDO A ARQUITETURA ORIENTADA A SERVIÇOS (SOA):

CONTEXTO E O QUE ELA NÃO É

1. O CONTEXTO: CRESCIMENTO DO TI E A BOLA DE PELO



2. A SOLUÇÃO: POR QUE SOA?

- Reuso de Funcionalidades
- Redução de Dependências
- Facilidade na Integração
- Flexibilidade e Sustentabilidade

3. O QUE É SOA E O QUE NÃO É

SOA É... UM PARADIGMA ARQUITETURAL

Conjunto de Princípios, Práticas e Padrões para organizar sistemas de forma Flexível.

INDEPENDÊNCIA TECNOLÓGICA:

Os princípios valem para qualquer plataforma de implementação.

SOA vs. WEB SERVICES

SOA = ARQUITETURA

WEB SERVICES = TECNOLOGIA DE IMPLEMENTAÇÃO
(ex: SOAP, REST, CORBA, RPC)

O QUE SOA NÃO É...

- NÃO É UMA TECNOLOGIA
- NÃO É UM PRODUTO
- NÃO É UM WEB SERVICE
- NÃO É UM PROJETO DE TI
- NÃO É UM SOFTWARE
- NÃO É UM FRAMEWORK
- NÃO É UMA METODOLOGIA
- NÃO É UMA SOLUÇÃO DE NEGÓCIO
- NÃO É UM MIDDLEWARE
- NÃO É UM SERVIÇO
- NÃO É UMA FERRAMENTA

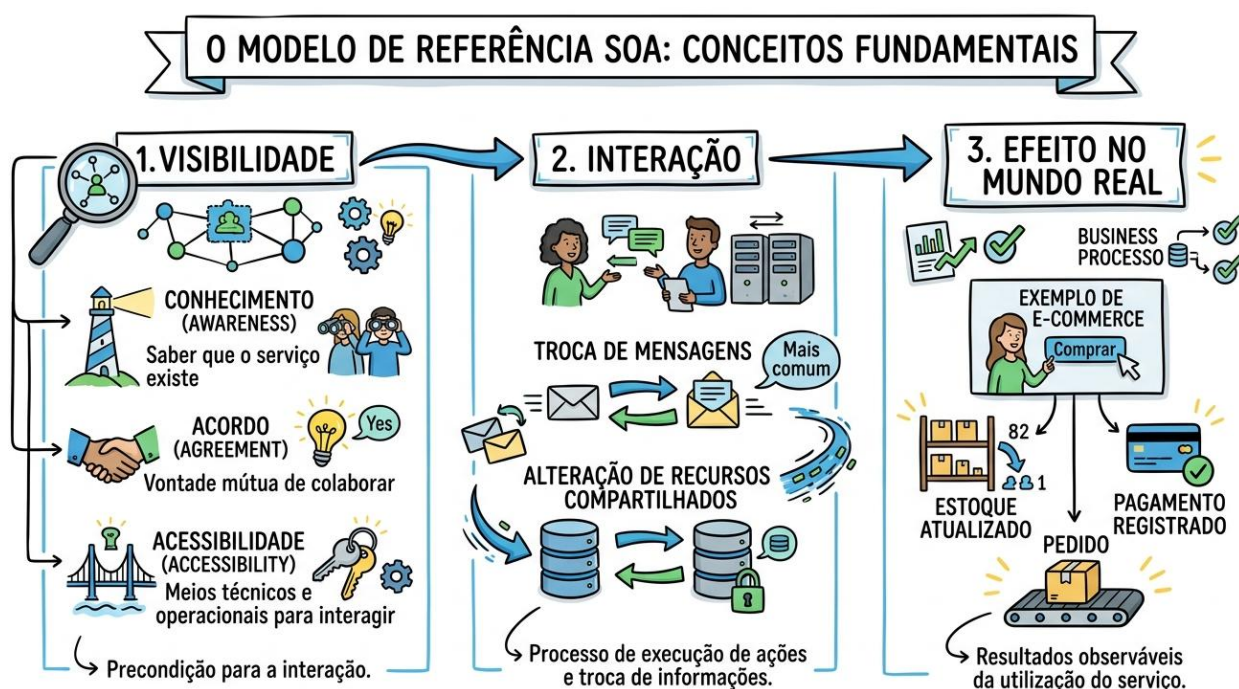
O Modelo de Referência para SOA descreve conceitos fundamentais relacionados à utilização de serviços e às interações entre consumidores e provedores. Entre seus elementos estão a descrição do serviço, contratos, políticas e contexto de execução. Esses componentes fornecem as informações e condições necessárias para que serviços possam ser descobertos, compreendidos e utilizados adequadamente, permitindo que sistemas distintos interajam de forma padronizada dentro de uma arquitetura orientada a serviços.

A descrição do serviço possui papel central nesse modelo, pois fornece informações que permitem aos consumidores avaliar se determinado serviço atende às suas necessidades. Além disso, favorece a descoberta e a interoperabilidade entre sistemas desenvolvidos de forma independente. Complementando esse mecanismo, contratos e políticas regulam a utilização dos serviços. Enquanto contratos representam acordos estabelecidos entre as partes, políticas definem restrições, condições ou requisitos que orientam a interação.

Outro conceito importante é o contexto de execução, que corresponde ao conjunto de elementos necessários para viabilizar uma interação de serviço. Esse contexto inclui infraestrutura tecnológica, processos, acordos, políticas e demais componentes envolvidos na operação. Em vez de representar apenas um canal de comunicação, o contexto descreve todo o ambiente operacional que sustenta a execução dos serviços e garante que as interações ocorram de forma adequada.

O modelo também destaca três conceitos essenciais para explicar a dinâmica das interações: visibilidade, interação e efeito no mundo real. A visibilidade representa a condição que permite que consumidores e provedores se relacionem, exigindo consciência mútua, concordância para interagir e acessibilidade aos mecanismos de comunicação. Uma vez estabelecida essa condição, a interação pode ocorrer por meio da troca de mensagens ou da manipulação de recursos compartilhados entre os participantes.

Toda interação gera um efeito no mundo real, conceito que representa os resultados concretos produzidos pela utilização dos serviços. Esses efeitos podem incluir alterações de estado, geração de informações ou impactos em processos de negócio. Em um comércio eletrônico, por exemplo, uma compra pode atualizar estoques, registrar pagamentos e criar pedidos. Assim, o Modelo de Referência para SOA demonstra como elementos estruturais e mecanismos de interação se combinam para produzir resultados observáveis e relevantes para o negócio.



Objetivos e Benefícios

INCIDÊNCIA EM PROVA: ALTA

Thomas Erl destaca que a Arquitetura Orientada a Serviços busca atingir objetivos estratégicos capazes de reduzir problemas comuns em ambientes corporativos. Entre os principais estão a interoperabilidade intrínseca, a federação e a diversificação de fornecedores. Esses objetivos procuram aumentar a capacidade de integração, reduzir dependências tecnológicas e tornar os sistemas mais flexíveis diante de mudanças organizacionais e tecnológicas, favorecendo a evolução contínua das soluções.

A interoperabilidade intrínseca representa a capacidade de os serviços serem concebidos desde sua origem para interagir com outros componentes de maneira padronizada. Em vez de tratar a integração como uma adaptação posterior, a orientação a serviços incorpora mecanismos que facilitam a comunicação entre sistemas desenvolvidos em diferentes linguagens, plataformas e tecnologias. Isso reduz a complexidade das integrações e torna o ambiente mais preparado para absorver novas soluções.

Outro objetivo importante é a federação, conceito que permite a colaboração entre recursos, aplicações e unidades organizacionais sem eliminar sua autonomia. Em um ambiente federado, cada área continua administrando seus próprios sistemas e processos, mas passa a compartilhar funcionalidades por meio de serviços padronizados. Dessa forma, torna-se possível promover integração e cooperação entre diferentes partes da organização sem exigir centralização completa ou uniformização tecnológica.

A federação é especialmente útil em organizações de grande porte, nas quais departamentos ou unidades de negócio possuem necessidades específicas. Ao permitir que cada participante preserve sua independência operacional, a arquitetura reduz conflitos entre demandas locais e objetivos corporativos. Assim, a organização consegue compartilhar informações, executar processos conjuntos e coordenar atividades de forma consistente, mantendo a flexibilidade necessária para atender particularidades de cada área.

A diversificação de fornecedores complementa esses benefícios ao reduzir a dependência de tecnologias e plataformas específicas. Por meio de serviços padronizados, as organizações ganham liberdade para avaliar diferentes provedores e substituir soluções quando necessário. Isso aumenta a resiliência da arquitetura, reduz riscos associados ao aprisionamento tecnológico e amplia a capacidade de adaptação diante de mudanças de mercado. Em conjunto, interoperabilidade, federação e diversificação tornam os ambientes mais integrados, flexíveis e sustentáveis.

ASPECTOS	VANTAGENS DO SOA	DESVANTAGENS DO SOA
REUTILIZAÇÃO	Serviços podem ser reutilizados por diferentes aplicações, reduzindo duplicação de funcionalidades.	O excesso de reutilização pode gerar dependências complexas entre sistemas.
INTEGRAÇÃO	Facilita a comunicação entre sistemas heterogêneos desenvolvidos em diferentes linguagens e plataformas.	A integração pode exigir camadas adicionais de transformação e adaptação de mensagens.
MANUTENÇÃO	Alterações em um serviço podem beneficiar diversos sistemas consumidores simultaneamente.	Mudanças em contratos de serviço podem impactar múltiplos consumidores.
ESCALABILIDADE	Serviços podem ser escalados de forma independente conforme a demanda.	A gestão de infraestrutura distribuída torna-se mais complexa.
FLEXIBILIDADE	Novas aplicações podem consumir serviços já existentes sem necessidade de reimplementação.	A governança dos serviços exige processos rigorosos de controle e versionamento.

ASPECTOS	VANTAGENS DO SOA	DESVANTAGENS DO SOA
DESENVOLVIMENTO	Equipes podem trabalhar de forma mais independente em serviços distintos.	Exige maior coordenação entre equipes para garantir compatibilidade e padronização.
INTEROPERABILIDADE	Utiliza padrões abertos (SOAP, XML, REST, JSON, etc), favorecendo a comunicação entre plataformas.	Diferenças de padrões e protocolos podem aumentar a complexidade da integração.
DISPONIBILIDADE	A falha de um serviço não necessariamente compromete toda a aplicação.	Dependências entre serviços podem provocar falhas em cascata.
DESEMPENHO	Permite distribuir processamento entre diversos serviços especializados.	A comunicação remota entre serviços adiciona latência e sobrecarga de rede.
GOVERNANÇA	Possibilita centralizar regras de negócio em serviços compartilhados.	Requer mecanismos de monitoramento, segurança, catálogo e gestão de serviços, aumentando o custo operacional.
SEGURANÇA	Permite implementar políticas de segurança padronizadas para os serviços.	Amplia a superfície de ataque, exigindo autenticação, autorização e criptografia bem estruturadas.
CUSTOS	Pode reduzir custos de desenvolvimento por meio do reaproveitamento de serviços.	A implantação inicial costuma demandar investimentos elevados em infraestrutura, ESB, monitoramento e governança.



Modelos de Serviços

MODELOS DE SERVIÇOS

Modelos de Serviços são representações conceituais que descrevem como os participantes de uma arquitetura orientada a serviços interagem para publicar, descobrir, acessar e consumir serviços. Esses modelos definem os papéis envolvidos – como provedores, consumidores e registros de serviços – bem como os mecanismos de comunicação e descoberta utilizados no ambiente SOA. Ao estabelecer padrões de interação, os modelos de serviços ajudam a organizar a infraestrutura de integração, promovendo interoperabilidade, reutilização, flexibilidade e desacoplamento entre os sistemas participantes da arquitetura.

Na Arquitetura Orientada a Serviços, não basta que serviços existam; é necessário que consumidores consigam localizá-los, compreendê-los e utilizá-los de forma organizada. Para atender essa necessidade, surgem os modelos de interação, descoberta e consumo, que descrevem como provedores, consumidores e mecanismos de registro se relacionam. Esses modelos estabelecem os fluxos necessários para publicação, localização e acesso aos serviços, permitindo que os participantes cooperem de maneira padronizada e eficiente.

Os modelos de interação complementam a arquitetura SOA ao detalhar os mecanismos de comunicação entre os participantes. Enquanto a arquitetura define princípios, papéis e características gerais do ambiente orientado a serviços, os modelos especificam como os serviços são disponibilizados, descobertos e consumidos. Diferentes abordagens podem ser adotadas conforme os objetivos da organização: algumas priorizam simplicidade e desempenho, enquanto outras enfatizam flexibilidade, reutilização, governança e baixo acoplamento, adequando-se a diferentes cenários corporativos.

Modelo Triangular

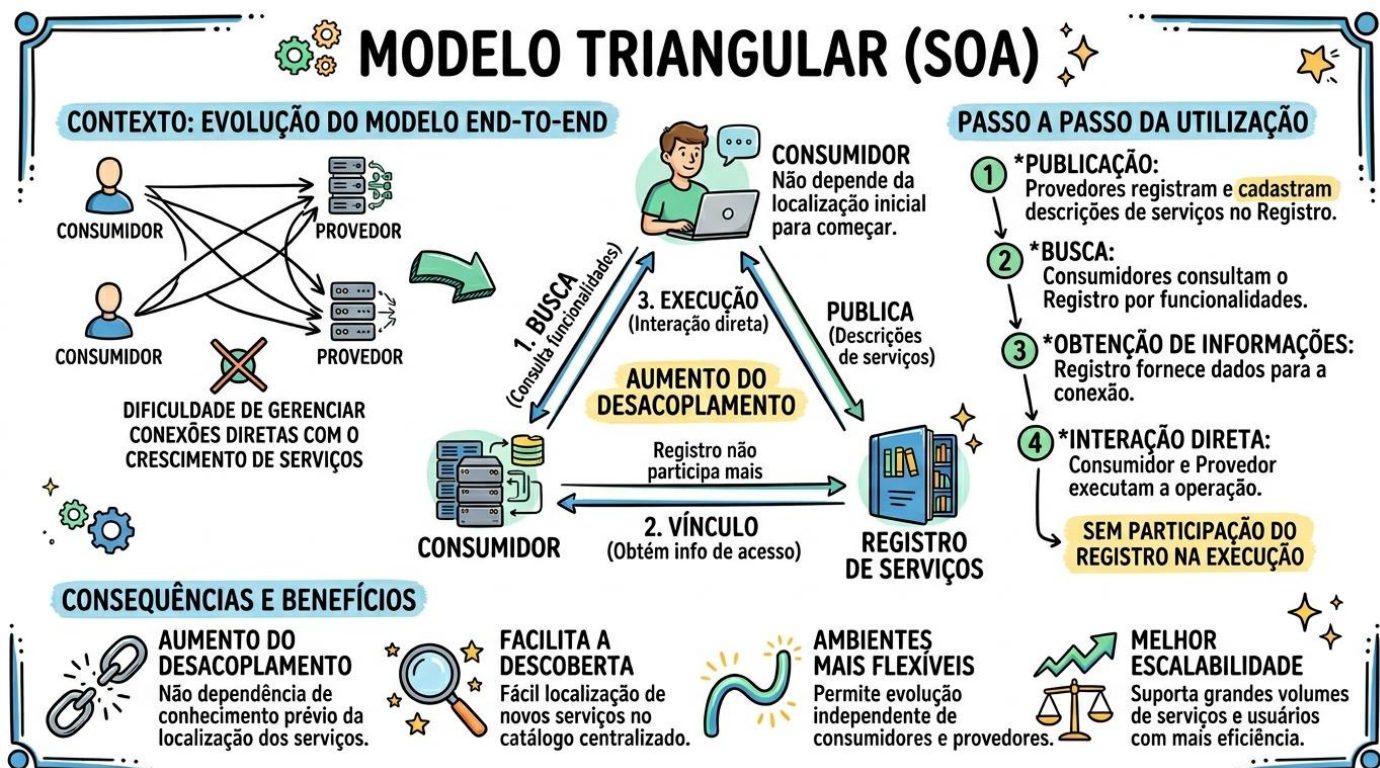
INCIDÊNCIA EM PROVA: MÉDIA

MODELO TRIANGULAR

O modelo triangular introduz um terceiro elemento na arquitetura: o registro de serviços. Assim, a interação ocorre entre três participantes principais – provedor, consumidor e registro – permitindo que os serviços sejam publicados e descobertos dinamicamente. Esse modelo aumenta a flexibilidade e a escalabilidade dos ambientes SOA ao desacoplar consumidores e provedores.

O modelo Triangular surgiu para solucionar limitações do modelo End-to-End em ambientes com grande quantidade de serviços. Nesse cenário, exigir que consumidores conheçam previamente todos os provedores torna-se inviável. Para resolver esse problema, o modelo introduz um terceiro participante: o registro de serviços. Esse componente funciona como um catálogo centralizado onde provedores publicam descrições de seus serviços, permitindo que consumidores localizem funcionalidades sem depender de conhecimento prévio sobre quem as oferece.

O processo ocorre em etapas: o consumidor consulta o registro em busca de um serviço compatível, obtém informações de acesso e, então, estabelece comunicação direta com o provedor. Após a descoberta, o registro não participa da execução da operação. Essa abordagem aumenta o desacoplamento entre os participantes, pois a localização dos serviços deixa de depender de configurações prévias. Como resultado, a descoberta de novos serviços é facilitada e o ambiente torna-se mais flexível, escalável e adaptável a mudanças.



Find-Bind-Execute

INCIDÊNCIA EM PROVA: BAIXA

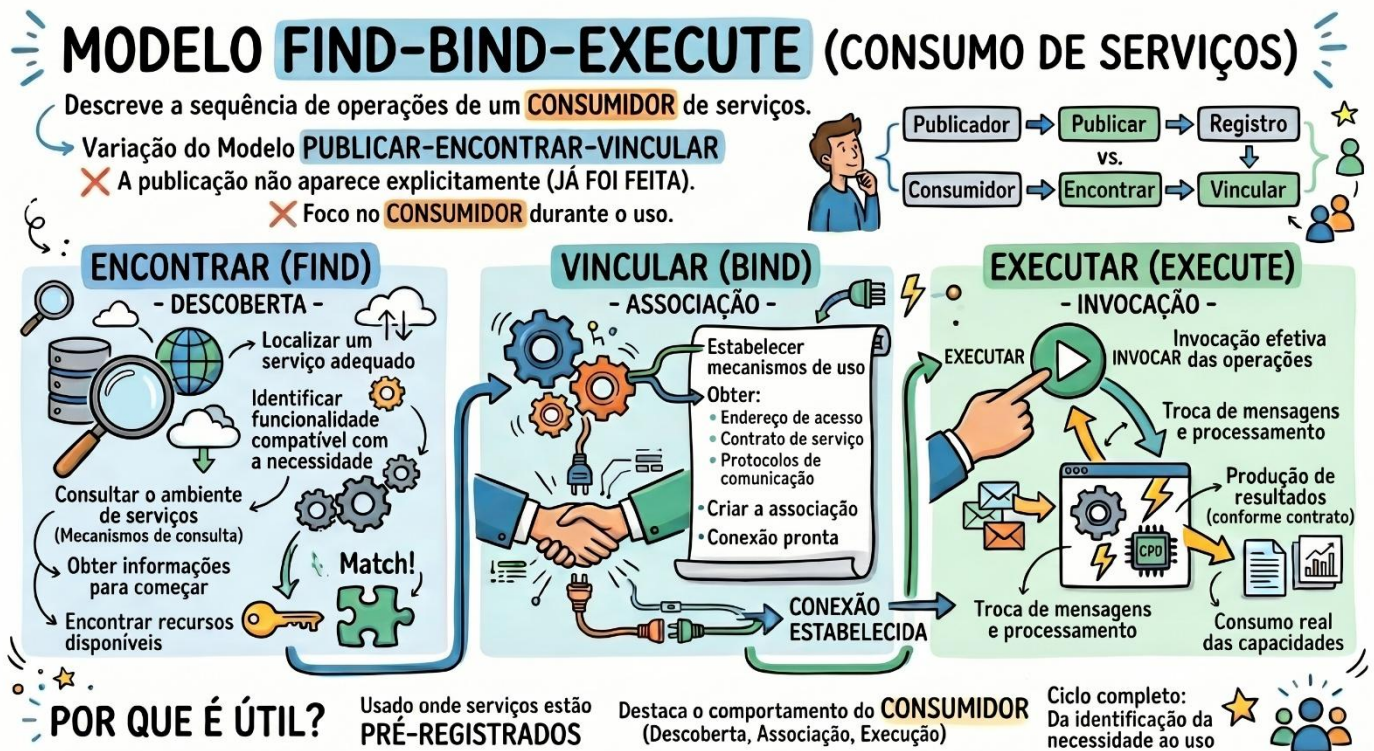
FIND-BIND-EXECUTE

O modelo Find-Bind-Execute descreve a sequência operacional utilizada pelo consumidor de serviços. Primeiro, o consumidor localiza um serviço adequado; em seguida, estabelece uma ligação com ele utilizando seu contrato; por último, executa as operações disponibilizadas pelo serviço. Esse fluxo representa o comportamento típico de consumo em arquiteturas orientadas a serviços.

O modelo Find-Bind-Execute pode ser visto como uma variação operacional do Publish-Find-Bind. A principal diferença é que a etapa de publicação não aparece explicitamente, pois se assume que os serviços já foram previamente registrados e disponibilizados no ambiente. Dessa forma, o modelo concentra-se nas ações realizadas pelo consumidor durante o processo de utilização do serviço, descrevendo o caminho percorrido desde a identificação de uma necessidade até o consumo efetivo das funcionalidades oferecidas.

A primeira fase é o Find, responsável pela descoberta do serviço. Nessa etapa, o consumidor realiza consultas para localizar uma funcionalidade compatível com seus requisitos, obtendo informações sobre os recursos disponíveis. Em seguida ocorre o Bind, momento em que são estabelecidos os elementos necessários para a comunicação, como endereço de acesso, contratos, protocolos e demais informações que permitem a associação entre consumidor e provedor de forma adequada e padronizada.

A etapa final é o Execute, que corresponde à utilização efetiva do serviço. Nesse momento, as operações são invocadas, as mensagens são trocadas e os resultados são produzidos conforme as definições estabelecidas pelo contrato. O modelo é especialmente útil para representar ambientes em que os serviços já estão disponíveis para consulta, destacando o comportamento do consumidor e evidenciando as etapas de descoberta, associação e execução que compõem o ciclo de consumo em uma arquitetura orientada a serviços.



Princípios De Design

INCIDÊNCIA EM PROVA: ALTÍSSIMA

PRINCÍPIOS DE DESIGN

Princípios de Design SOA são diretrizes arquiteturais que orientam a criação de serviços de forma consistente, reutilizável e alinhada aos objetivos da Arquitetura Orientada a Serviços. Esses princípios estabelecem características desejáveis para os serviços, como contratos padronizados, baixo acoplamento, abstração, autonomia, reusabilidade, independência de estado, visibilidade e capacidade de composição. Ao serem aplicados durante a modelagem e implementação, contribuem para a construção de soluções mais flexíveis, interoperáveis, escaláveis e fáceis de manter, aumentando o potencial de integração e reutilização dos serviços em diferentes processos e aplicações da organização.

Thomas Erl é um dos autores mais influentes no estudo da Arquitetura Orientada a Serviços. Suas obras contribuíram para consolidar conceitos, práticas e diretrizes amplamente adotados na modelagem de serviços. Em vez de focar em tecnologias específicas, Erl propõe princípios de design que orientam a construção de serviços mais reutilizáveis, interoperáveis, flexíveis e alinhados aos objetivos de negócio. Por essa razão, seus trabalhos são amplamente utilizados em cursos, certificações e iniciativas de adoção de SOA.

Os princípios de design de serviços têm como finalidade orientar decisões arquiteturais relacionadas à estrutura e ao comportamento dos serviços. Eles procuram garantir que os serviços possam ser utilizados por diferentes consumidores, evoluir ao longo do tempo e manter sua utilidade dentro da organização. Para isso, enfatizam características como contratos bem definidos, autonomia, baixo acoplamento e outras práticas que favorecem a manutenção e a integração em ambientes orientados a serviços.

Entre os conceitos associados ao design de serviços, a granularidade ocupa posição de destaque. Ela está relacionada à abrangência funcional de um serviço ou de suas operações, procurando definir quanto de funcionalidade deve ser agrupado em uma determinada unidade de serviço. A decisão sobre a granularidade influencia diretamente a forma como as capacidades de negócio são disponibilizadas e consumidas dentro da arquitetura, tornando-se um aspecto fundamental da modelagem orientada a serviços.

A definição inadequada da granularidade pode gerar problemas significativos. Serviços excessivamente detalhados tendem a exigir muitas chamadas para executar um processo de negócio, aumentando a complexidade da comunicação. Por outro lado, serviços excessivamente amplos podem reduzir oportunidades de reutilização e dificultar a manutenção. Por isso, a granularidade exige um equilíbrio cuidadoso entre eficiência operacional, simplicidade arquitetural e potencial de reaproveitamento das funcionalidades disponibilizadas.

É importante diferenciar granularidade de outros princípios frequentemente estudados em SOA. O baixo acoplamento busca reduzir dependências entre serviços, permitindo maior autonomia e facilidade de evolução. Já a ausência de estado procura minimizar a retenção de informações entre requisições, favorecendo escalabilidade e flexibilidade. A granularidade, por sua vez, concentra-se exclusivamente na definição do escopo funcional dos serviços, embora possa influenciar indiretamente os demais aspectos arquiteturais.

A escolha da granularidade ideal depende dos processos de negócio, dos requisitos da solução e dos objetivos estabelecidos para a arquitetura. Não existe uma regra universal capaz de determinar o tamanho adequado de todos os serviços. Nos estudos de Thomas Erl, a granularidade é analisada em conjunto com outros princípios clássicos de design, que atuam de forma complementar para promover

reutilização, interoperabilidade, flexibilidade e capacidade de evolução dentro de uma Arquitetura Orientada a Serviços.

≡ PRINCÍPIOS DE DESIGN PARA SERVIÇOS ≡



Contrato de Serviço Padronizado (*Service Contract*)

O princípio do Contrato de Serviço Padronizado estabelece que os serviços devem disponibilizar contratos claros e consistentes, contendo informações necessárias para sua utilização. Esses contratos descrevem operações disponíveis, formatos de mensagens, requisitos, políticas e demais elementos que orientam a comunicação entre consumidores e provedores. Seu objetivo é tornar os serviços compreensíveis, permitindo que diferentes consumidores identifiquem facilmente como acessar e utilizar as funcionalidades disponibilizadas.

Mais do que simplesmente existir, o contrato deve seguir padrões comuns adotados pela organização. A padronização reduz diferenças desnecessárias entre interfaces, facilita a descoberta dos serviços e aumenta a previsibilidade das integrações. Por meio do contrato, os consumidores conseguem avaliar se determinado serviço atende às suas necessidades, compreender quais dados devem ser fornecidos e identificar os resultados esperados, fortalecendo a interoperabilidade e a reutilização dentro da arquitetura.

Os contratos também podem incluir ou referenciar informações complementares, como níveis de serviço, requisitos operacionais e responsabilidades das partes. Como os serviços evoluem ao longo do tempo, seus contratos frequentemente precisam ser atualizados para refletir novas funcionalidades ou ajustes realizados. Nesse contexto, o versionamento torna-se uma atividade importante da governança de serviços, ajudando a preservar compatibilidade entre consumidores e provedores e garantindo a consistência do ambiente orientado a serviços.

Baixo Acoplamento de Serviços (*Service Loose Coupling*)

O princípio do Baixo Acoplamento busca reduzir ao máximo as dependências entre serviços, sistemas e componentes externos. Quanto menor for a dependência entre os participantes da arquitetura, maior será a capacidade de modificar, substituir ou evoluir serviços sem provocar impactos significativos nos demais elementos do ambiente. Essa característica favorece a flexibilidade, a manutenção e a evolução contínua da solução, sendo considerada um dos pilares fundamentais da Arquitetura Orientada a Serviços.

O conceito de acoplamento está intimamente relacionado à coesão. Enquanto o acoplamento mede o grau de dependência entre componentes, a coesão avalia o quanto as responsabilidades de um serviço estão relacionadas entre si. Serviços altamente coesos concentram funcionalidades ligadas a um mesmo objetivo de negócio. Quando um serviço reúne responsabilidades muito diferentes, sua manutenção tende a se tornar mais difícil e seu potencial de reutilização pode ser reduzido.

Outro aspecto importante associado ao design de serviços é a granularidade, que define a abrangência das funcionalidades disponibilizadas por cada serviço. Essa decisão influencia diretamente fatores como reutilização, desempenho, composição e facilidade de manutenção. Em essência, a granularidade procura determinar quanto de responsabilidade deve ser atribuído a cada serviço, estabelecendo as fronteiras funcionais dentro da arquitetura orientada a serviços.

Os serviços de granularidade fina, conhecidos como fine-grained, concentram-se em funcionalidades específicas e bem delimitadas. Essa especialização normalmente favorece a reutilização, pois pequenas capacidades podem ser combinadas em diferentes cenários. Entretanto, a fragmentação excessiva pode aumentar a quantidade de chamadas necessárias para executar processos de negócio, elevando a complexidade das integrações e gerando impactos relacionados à latência e ao tráfego de rede.

Já os serviços de granularidade grossa, ou coarse-grained, oferecem funcionalidades mais amplas e frequentemente representam processos de negócio mais completos. Essa abordagem pode reduzir a necessidade de múltiplas interações entre serviços e simplificar o consumo das funcionalidades. Contudo, serviços excessivamente abrangentes tendem a dificultar a reutilização e podem concentrar responsabilidades demais, tornando a manutenção e a evolução mais complexas ao longo do tempo.

Por essa razão, a literatura de SOA enfatiza a busca por uma granularidade adequada, equilibrando reutilização, desempenho e simplicidade arquitetural. Não existe uma regra universal para definir o tamanho ideal dos serviços, pois essa decisão depende dos requisitos do negócio e da arquitetura adotada. Embora o desacoplamento absoluto seja inviável na prática, tecnologias como o Enterprise Service Bus (ESB) foram criadas para facilitar a comunicação entre serviços e reduzir dependências diretas entre os participantes.

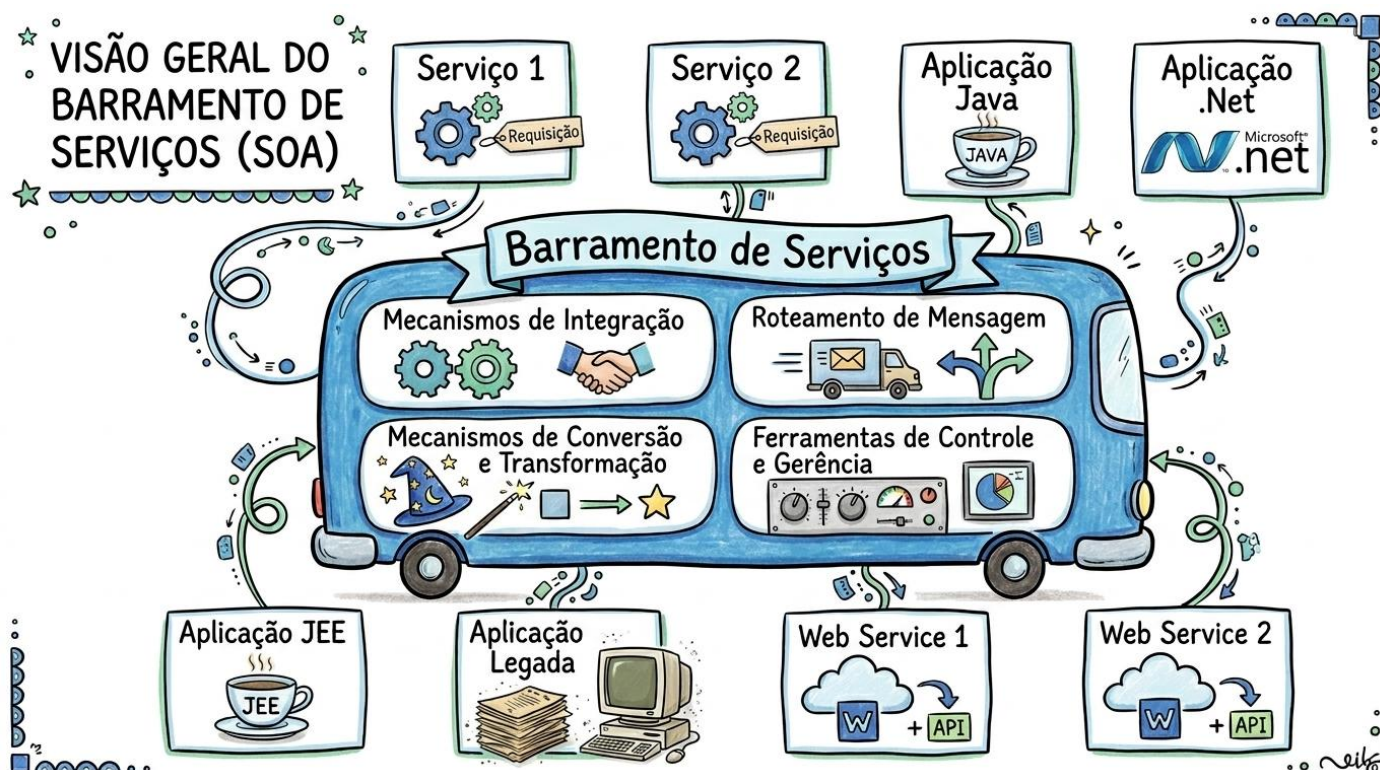
O Enterprise Service Bus (ESB) é uma tecnologia de integração amplamente associada aos ambientes SOA. Sua principal função é facilitar a comunicação entre aplicações desenvolvidas em diferentes linguagens, plataformas e tecnologias. Em vez de exigir conexões diretas entre todos os sistemas participantes, o barramento atua como uma camada intermediária de integração. Essa abordagem reduz o acoplamento entre aplicações e torna a arquitetura mais flexível para acomodar mudanças e expansões futuras.

Uma analogia comum consiste em comparar o ESB a uma rede de transporte. Assim como diferentes destinos podem ser alcançados por meio de uma infraestrutura compartilhada, os sistemas trocam informações utilizando o barramento como intermediário. Os serviços permanecem concentrados em suas responsabilidades de negócio, enquanto a infraestrutura de integração assume atividades relacionadas ao encaminhamento, tratamento e coordenação das mensagens trocadas entre os participantes do ambiente.

Entre os recursos frequentemente oferecidos por um ESB estão o roteamento de mensagens, a transformação de dados, a conversão de protocolos, a resolução de descrições de serviços, o tratamento de exceções e o monitoramento das integrações. Essas capacidades permitem que sistemas heterogêneos se comuniquem mesmo quando utilizam formatos de dados ou tecnologias incompatíveis. Dessa forma, reduz-se a necessidade de implementar individualmente toda a lógica de integração em cada aplicação envolvida.

A comunicação com o barramento ocorre por meio de endpoints, que funcionam como pontos de entrada e saída das mensagens. Embora sejam responsáveis por estabelecer a conexão entre aplicações e infraestrutura de integração, os endpoints nem sempre executam toda a lógica de processamento. Dependendo da solução adotada, funções como transformação, validação, monitoramento e roteamento podem ser realizadas por outros componentes especializados da plataforma de integração.

Além de facilitar a comunicação, muitas soluções de ESB oferecem recursos de auditoria, rastreabilidade e monitoramento operacional, permitindo acompanhar o fluxo das mensagens e identificar problemas com maior facilidade. Apesar de sua forte associação com SOA, o ESB não é um requisito obrigatório para a implementação da arquitetura orientada a serviços. Trata-se de uma tecnologia que pode apoiar a integração de sistemas, mas SOA e ESB permanecem conceitos distintos e independentes.



Abstração de Serviços (Service Abstraction)

O princípio da Abstração de Serviços determina que consumidores utilizem os serviços sem depender do conhecimento de sua implementação interna. De forma semelhante aos testes caixa-preta, o foco está no comportamento observável e nas funcionalidades disponibilizadas, e não nos mecanismos utilizados para executá-las. Assim, os consumidores interagem por meio dos contratos de serviço, acessando apenas as informações necessárias para utilização adequada dos recursos oferecidos pelo provedor.

Uma das principais vantagens da abstração é permitir que a implementação evolua sem afetar os consumidores. Alterações internas, otimizações, correções ou refatorações podem ser realizadas

livremente desde que o contrato e o comportamento esperado permaneçam consistentes. Dessa forma, a dependência dos consumidores concentra-se nas interfaces expostas, e não nas tecnologias, estruturas ou decisões técnicas utilizadas para implementar as funcionalidades disponibilizadas pelo serviço.

A abstração também orienta a definição dos contratos de serviço, que devem expor apenas informações relevantes para o consumo, como operações disponíveis, formatos de mensagens e políticas aplicáveis. Detalhes internos permanecem encapsulados dentro dos limites do serviço. Esse equilíbrio evita tanto a exposição excessiva de informações quanto a simplificação exagerada dos contratos. Como resultado, os serviços tornam-se mais flexíveis, mais fáceis de manter e menos vulneráveis a impactos causados por mudanças internas.

Reusabilidade de Serviços (Service Reusability)

A Reusabilidade de Serviços é um princípio que busca criar serviços capazes de ser utilizados em diferentes aplicações, processos de negócio e contextos organizacionais. Em vez de atender apenas uma necessidade específica, o serviço é projetado para disponibilizar capacidades que possam ser aproveitadas repetidamente por diversos consumidores. Essa característica transforma os serviços em ativos reutilizáveis, ampliando seu valor dentro da arquitetura orientada a serviços.

O potencial de reutilização depende diretamente da forma como os serviços são modelados. Serviços excessivamente vinculados a uma aplicação, tecnologia ou processo específico tendem a apresentar menor capacidade de reaproveitamento. Por outro lado, quando as funcionalidades são definidas de maneira mais independente e orientadas a capacidades de negócio, aumentam as possibilidades de utilização em diferentes projetos e iniciativas organizacionais ao longo do tempo.

A reutilização também é favorecida por contratos claros e por outros princípios de design de serviços. Contratos padronizados facilitam a descoberta e a compreensão das capacidades disponíveis, enquanto características como baixo acoplamento e abstração reduzem dependências desnecessárias. Dessa forma, o reúso não resulta de um único fator isolado, mas da combinação de diversas decisões arquiteturais que tornam os serviços mais acessíveis e adaptáveis a diferentes cenários.

Projetar serviços reutilizáveis normalmente exige maior esforço inicial, pois demanda análise de múltiplos contextos de utilização e a identificação de capacidades mais amplas e duradouras. Entretanto, esse investimento tende a gerar benefícios significativos, como redução da duplicação de funcionalidades, menor custo de desenvolvimento e maior velocidade na criação de novas soluções. O objetivo não é criar serviços excessivamente genéricos, mas disponibilizar capacidades úteis, bem definidas e aplicáveis em diversos contextos organizacionais.

Autonomia de Serviços (Service Autonomy)

O princípio da Autonomia de Serviços estabelece que cada serviço deve possuir o maior controle possível sobre sua lógica de negócio e sobre os recursos necessários para sua execução. Um serviço autônomo consegue desempenhar suas responsabilidades com menor influência de fatores externos, tornando seu comportamento mais previsível. O objetivo não é alcançar independência absoluta, mas aumentar a capacidade do serviço de governar suas próprias operações e reduzir vulnerabilidades associadas a dependências excessivas.

Na prática, muitos serviços dependem de bancos de dados, sistemas externos, filas de mensagens e outros componentes da infraestrutura. Por esse motivo, a autonomia não significa eliminar todas as dependências, algo normalmente inviável em ambientes corporativos. O princípio procura reduzir

dependências que possam comprometer a estabilidade ou limitar o controle do serviço sobre sua execução, favorecendo uma operação mais consistente e menos suscetível a interferências externas.

A autonomia também está relacionada ao ambiente operacional. Serviços que exercem maior controle sobre os recursos utilizados durante sua execução tendem a apresentar maior confiabilidade e estabilidade. Como dependem menos de elementos externos para realizar suas funções, ficam menos expostos a falhas causadas por mudanças em outros componentes da arquitetura. Além disso, a redução de pontos de dependência pode contribuir para melhorar a eficiência e diminuir interrupções durante o processamento.

Outro benefício importante é o aumento da previsibilidade operacional. Quando um serviço controla melhor os fatores que influenciam seu funcionamento, atividades como monitoramento, manutenção, diagnóstico de falhas e planejamento de capacidade tornam-se mais simples. Assim, a autonomia busca criar serviços capazes de administrar suas próprias capacidades operacionais, promovendo maior confiabilidade, estabilidade e consistência na execução das funcionalidades disponibilizadas pela Arquitetura Orientada a Serviços.

Independência de Estados de Serviços (Service Statelessness)

O princípio da Independência de Estados de Serviços determina que os serviços devem minimizar a retenção de informações de contexto entre diferentes interações. Um serviço stateless procura processar cada requisição utilizando apenas os dados fornecidos naquele momento, reduzindo a necessidade de armazenar informações relacionadas a sessões anteriores. O objetivo não é eliminar completamente o estado, mas evitar que ele permaneça desnecessariamente mantido dentro do próprio serviço.

Na prática, muitos sistemas precisam registrar dados relacionados a usuários, transações e processos de negócio. Entretanto, essas informações podem ser armazenadas em mecanismos especializados de persistência, em vez de permanecerem vinculadas ao ciclo de vida do serviço. Esse conceito é frequentemente ilustrado pelo protocolo HTTP, no qual cada requisição contém os dados necessários para seu processamento, permitindo que novas solicitações sejam tratadas de forma independente das anteriores.

A redução da dependência de estado produz benefícios importantes para a escalabilidade. Como os serviços não precisam manter grandes quantidades de informações associadas a conexões ou sessões ativas, torna-se mais simples distribuir requisições entre diferentes instâncias de execução. Essa característica facilita o balanceamento de carga, aumenta a capacidade de atendimento simultâneo e contribui para ambientes distribuídos mais flexíveis e preparados para crescimento.

Além da escalabilidade, a independência de estado favorece o uso eficiente dos recursos computacionais e aumenta a previsibilidade do comportamento dos serviços. A menor necessidade de manter memória e conexões reservadas reduz a complexidade operacional e melhora o aproveitamento da infraestrutura. Como consequência, diminuem os riscos de inconsistências relacionadas a sessões ou estados temporários, promovendo maior confiabilidade, desempenho e estabilidade dentro da Arquitetura Orientada a Serviços.

Visibilidade de Serviços (Service Discoverability)

O princípio da Visibilidade de Serviços, ou Service Discoverability, estabelece que os serviços devem ser projetados para serem facilmente localizados e compreendidos por potenciais consumidores. Em uma arquitetura orientada a serviços, não basta que uma funcionalidade exista; é necessário que sua presença

e suas capacidades sejam conhecidas. Quanto mais fácil for descobrir um serviço e entender seu propósito, maiores serão as oportunidades de reutilização dentro da organização.

A descoberta está diretamente relacionada à disponibilização de informações adequadas sobre o serviço. Além dos contratos técnicos, elementos como descrições claras, documentação consistente, classificações e metadados contribuem para que consumidores identifiquem rapidamente as funcionalidades oferecidas. Essas informações permitem compreender a finalidade do serviço e avaliar sua adequação às necessidades existentes sem exigir conhecimento sobre detalhes internos de implementação.

A visibilidade não se limita à localização dos serviços. Após encontrar um serviço, o consumidor precisa compreender suas capacidades de maneira clara e objetiva. Por esse motivo, a descobribilidade envolve tanto a facilidade de encontrar quanto a qualidade das informações disponibilizadas. Um serviço mal documentado ou descrito de forma inadequada pode continuar sendo pouco utilizado, mesmo estando tecnicamente acessível dentro do ambiente orientado a serviços.

A preocupação com a descobribilidade deve estar presente desde a modelagem dos serviços, garantindo consistência entre contratos, documentação e funcionalidades implementadas. Um dos principais benefícios desse princípio é a redução da redundância de soluções, pois facilita a identificação de funcionalidades já existentes. Como resultado, aumenta-se o reuso dos serviços, reduz-se a duplicação de esforços e promove-se um melhor aproveitamento dos ativos de software disponíveis na arquitetura.

Composição de Serviços (Service Composability)

Serviços são projetados para que atuem como participantes eficazes de uma composição, independentemente do tamanho e da complexidade de composição. Ao discutir os objetivos do princípio da composição de serviços, boa parte dos objetivos de reuso de serviços também são aplicáveis. Isso ocorre porque, muitas vezes, a composição de serviços é uma forma de reutilização de serviços.

Talvez você se lembre que um dos objetivos que selecionamos para o princípio da reusabilidade era possibilitar a composição de serviços em larga escala. A composição de serviços permite que as capacidades de um serviço sejam combinadas várias vezes com as de outros serviços em novas configurações a fim de resolver problemas diferentes. *Vamos ver um exemplo?* Os correios disponibilizam um serviço tal que, dado um CEP, retorna um endereço.

Eles também oferecem outro serviço que – dado um CEP – retorna os tipos de frete e seus valores. Esses serviços isoladamente resolvem problemas específicos. Quando compostos, esses resolvem outros tipos de problemas. Ao estabelecer em uma empresa uma lógica representada por um inventário de serviços altamente reusáveis, fornecemos o meio para que uma grande extensão dos futuros requisitos da automação de negócios seja cumprida por meio da composição de serviços.

Esse princípio trata de serviços se juntarem e serem acessados para englobar e atender um problema maior. Estas características se relacionam de forma interdependente. Por exemplo: o *baixo acoplamento* reduz a *alocação de recursos*; *contrato padronizado* forma as bases para o *descobrimento*; *alocação de recursos* reduzida maximiza de *reutilização*; *descobrimento* promove *reutilização*; *autonomia* reduz a *alocação de recursos*; *baixo acoplamento* permite *autonomia*.

PRINCÍPIOS DE DESIGN	DESCRIÇÃO
CONTRATO PADRONIZADO	O princípio do Contrato Padronizado estabelece que os serviços devem expor suas capacidades por meio de contratos bem definidos, consistentes e compreensíveis. Esses

	contratos descrevem operações, formatos de mensagens, políticas e requisitos de interação, permitindo interoperabilidade entre diferentes sistemas.
BAIXO ACOPLAMENTO	O baixo acoplamento determina que os serviços mantenham o mínimo possível de dependências entre si. Dessa forma, alterações internas em um serviço tendem a produzir impactos reduzidos nos demais componentes da arquitetura, favorecendo manutenção e evolução independente.
ABSTRAÇÃO DE SERVIÇOS	A abstração estabelece que detalhes internos de implementação devem permanecer ocultos aos consumidores. Os usuários do serviço precisam conhecer apenas o contrato e as funcionalidades disponibilizadas, sem depender da lógica interna ou da tecnologia utilizada.
REUSABILIDADE	A reusabilidade orienta a construção de serviços capazes de atender múltiplos consumidores e diferentes processos de negócio. Quanto maior o potencial de reutilização, menor a duplicação de funcionalidades e maior o retorno sobre os investimentos realizados na arquitetura.
AUTONOMIA DE SERVIÇOS	A autonomia determina que cada serviço possua controle sobre sua própria lógica, recursos e ambiente de execução. Serviços autônomos reduzem dependências externas e aumentam a capacidade de evolução, manutenção e implantação independente.
INDEPENDÊNCIA DE ESTADOS	A independência de estado recomenda que os serviços mantenham o mínimo possível de informações sobre interações anteriores. Essa característica reduz dependências temporais entre requisições, facilita escalabilidade e melhora a tolerância a falhas.
VISIBILIDADE DE SERVIÇOS	O princípio da descoberta estabelece que os serviços devem ser facilmente localizados e compreendidos pelos potenciais consumidores. Para isso, utilizam descrições padronizadas, registros de serviços e documentação adequada que favorecem sua reutilização.
COMPOSIÇÃO DE SERVIÇOS	A composabilidade determina que os serviços sejam projetados para participar facilmente da construção de soluções maiores. Serviços composáveis podem ser combinados em fluxos, processos ou serviços compostos, ampliando sua capacidade de reutilização.

Composição De Serviços

INCIDÊNCIA EM PROVA: ALTA

COMPOSIÇÃO DE SERVIÇOS

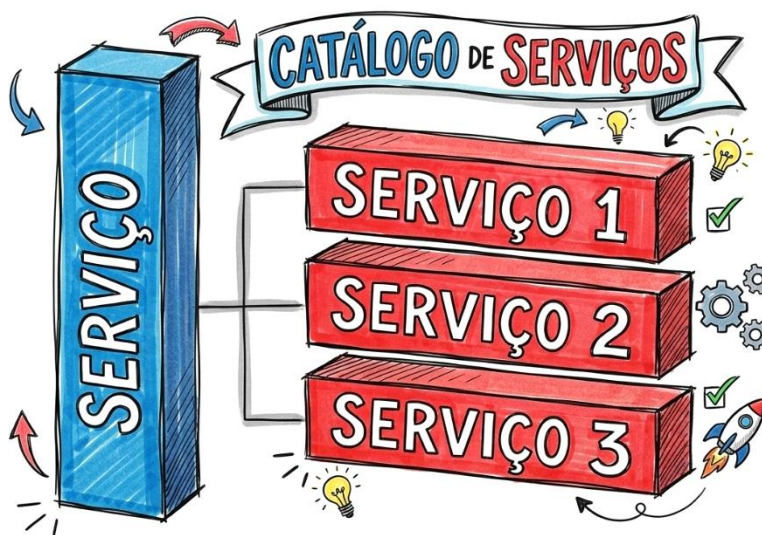
A composição de serviços consiste na combinação de múltiplos serviços independentes para formar soluções mais complexas e alinhadas aos processos de negócio. Essa abordagem permite criar funcionalidades avançadas reutilizando capacidades já existentes na arquitetura.

O princípio da Composição de Serviços estabelece que funcionalidades mais complexas podem ser construídas por meio da combinação de serviços menores e especializados. Em vez de concentrar toda a lógica em um único componente, diferentes capacidades são organizadas para atuar em conjunto, formando serviços compostos ou processos de negócio mais abrangentes. Essa abordagem aumenta a modularidade da arquitetura e permite criar soluções mais sofisticadas a partir de recursos já existentes.

A composição é um dos principais mecanismos para promover reutilização em SOA. Um mesmo serviço pode participar de diferentes processos de negócio e ser combinado de diversas maneiras conforme as necessidades da organização evoluem. Isso reduz a necessidade de desenvolver novas funcionalidades do zero e amplia o aproveitamento dos ativos de software existentes. Como resultado, a arquitetura ganha flexibilidade e maior capacidade de adaptação a mudanças de negócio.

As interações entre serviços compostos podem ocorrer de forma síncrona ou assíncrona, dependendo dos requisitos do processo. Além disso, a composição não elimina o princípio da Independência de Estados. Os serviços individuais continuam sendo projetados para minimizar a retenção de informações entre requisições. O estado necessário para acompanhar a execução do processo é normalmente mantido por mecanismos de coordenação, workflow ou orquestração responsáveis pelo controle do fluxo de atividades.

Para garantir que cada serviço seja executado no momento adequado e que as informações circulem corretamente, utilizam-se mecanismos especializados de coordenação e gerenciamento de processos. Esses recursos controlam dependências, acompanham o progresso das atividades e administram o estado do processo composto. Dessa forma, a composição permite construir soluções complexas preservando reutilização, flexibilidade e modularidade, tornando a arquitetura mais preparada para evoluir e atender novas demandas organizacionais.



Orquestração de Serviços

ORQUESTRAÇÃO DE SERVIÇOS

A orquestração é uma abordagem centralizada de composição em que um coordenador controla a sequência de execução dos serviços participantes. Esse elemento central define o fluxo do processo, decide quais serviços serão invocados e gerencia aspectos como transações, exceções e transformações de dados.

A orquestração de serviços é um mecanismo de composição no qual um componente central coordena a execução de um processo de negócio composto. Em vez de cada serviço decidir autonomamente quando interagir com os demais, existe uma entidade responsável por controlar o fluxo das atividades. Essa abordagem permite combinar diversos serviços independentes em uma solução mais ampla, organizada e alinhada aos objetivos do negócio, mantendo a coordenação das operações em um único ponto de controle.

O elemento central da orquestração é o coordenador, responsável por administrar a execução do processo. Ele recebe solicitações, invoca serviços, processa resultados intermediários e determina quais atividades devem ser executadas em seguida. Como possui uma visão completa do fluxo de negócio, consegue controlar dependências, aplicar regras de decisão e garantir que cada etapa seja realizada no momento apropriado, promovendo maior previsibilidade na execução do processo composto.

Uma analogia clássica compara a orquestração a uma orquestra musical. Os serviços individuais correspondem aos músicos, cada um especializado em determinada função. O processo de negócio representa a apresentação completa, enquanto o coordenador exerce o papel do maestro. Assim como o maestro define quando cada instrumento deve atuar para produzir uma execução harmoniosa, o coordenador determina a sequência das interações entre os serviços para alcançar o resultado esperado.

Uma das principais vantagens da orquestração é a centralização da lógica do processo. Regras de negócio, fluxos de decisão, tratamentos de exceção e mecanismos de compensação podem ser definidos em um único local. Isso simplifica a administração de processos complexos e facilita atividades relacionadas ao monitoramento, à manutenção e à governança. Além disso, torna mais simples implementar controles associados a falhas, transações distribuídas e recuperação de erros.

Outro aspecto importante envolve o gerenciamento do estado do processo composto. Embora os serviços individuais possam permanecer stateless, o coordenador mantém informações sobre o andamento da execução, registrando etapas concluídas, atividades pendentes e ações necessárias diante de falhas ou interrupções. Dessa forma, o estado do processo é administrado centralmente, permitindo que os serviços permaneçam focados exclusivamente em suas responsabilidades específicas.

Entre as tecnologias mais associadas à orquestração em ambientes SOA destaca-se a WS-BPEL (Web Services Business Process Execution Language). Essa linguagem foi criada para modelar e executar processos compostos por Web Services utilizando estruturas baseadas em XML. Seu objetivo é fornecer mecanismos capazes de representar fluxos de atividades, regras de controle, condições, variáveis e interações entre serviços participantes de um mesmo processo de negócio.

A WS-BPEL disponibiliza diversas atividades fundamentais para a construção desses processos. O elemento <receive> é utilizado para receber mensagens e iniciar ou continuar a execução do fluxo. O <assign> permite copiar e manipular dados entre variáveis e mensagens. Já o <invoke> realiza chamadas a serviços parceiros, podendo operar de forma síncrona ou assíncrona conforme o padrão de troca de mensagens definido para a interação.

Outra atividade importante é o <reply>, responsável por retornar respostas aos participantes que iniciaram determinada solicitação. Em conjunto, elementos como <receive>, <assign>, <invoke> e <reply> permitem modelar processos complexos de forma estruturada. Assim, a orquestração combina serviços independentes sob controle centralizado, enquanto a WS-BPEL fornece os recursos necessários para implementar essa coordenação de maneira organizada, previsível e alinhada às necessidades da organização.



Coreografia de Serviços

COREOGRAFIA DE SERVIÇOS

A coreografia representa uma abordagem descentralizada de composição de serviços. Não existe um coordenador central; cada participante conhece previamente seu papel e interage diretamente com os demais seguindo regras previamente definidas. O comportamento global emerge da colaboração entre os serviços envolvidos.

A coreografia e a orquestração representam abordagens distintas para coordenar serviços em uma Arquitetura Orientada a Serviços. Na orquestração existe um coordenador central responsável por controlar o fluxo do processo e determinar quando cada serviço deve ser executado. Já na coreografia não há um controlador central; cada participante segue regras previamente definidas de comunicação, enviando e recebendo mensagens conforme suas responsabilidades. Assim, a coordenação surge das interações distribuídas entre os serviços.

A principal diferença entre os dois modelos está na localização da lógica de coordenação. Na orquestração, o controle do processo encontra-se concentrado em uma entidade central. Na coreografia, cada serviço conhece apenas as interações relacionadas ao seu papel e atua de forma autônoma dentro do protocolo estabelecido. O resultado é um modelo mais distribuído, no qual a colaboração ocorre sem que exista um componente responsável por controlar integralmente todas as etapas do processo.

A troca de mensagens assume papel fundamental na coreografia. Como não existe um coordenador central, a execução correta depende do cumprimento das regras de interação previamente definidas. Cada participante reage às mensagens recebidas e produz novas mensagens conforme o protocolo acordado. Dessa forma, a coordenação não é imposta por um único componente, mas emerge do comportamento coletivo dos serviços envolvidos na colaboração distribuída.

No universo dos Web Services, a linguagem WS-CDL (Web Services Choreography Description Language) foi criada para descrever essas colaborações. Seu objetivo era representar, sob uma perspectiva global, os participantes envolvidos, as mensagens trocadas e a sequência esperada das interações. Embora sua adoção prática tenha sido limitada, seus conceitos contribuíram para consolidar a distinção entre coordenação centralizada, típica da orquestração, e coordenação distribuída, característica da coreografia.

Outro tema relacionado à SOA é a Service Component Architecture (SCA), iniciativa apoiada por fornecedores como IBM, Oracle e SAP. O modelo foi concebido para facilitar o desenvolvimento de soluções orientadas a serviços por meio da composição de componentes reutilizáveis. Esses componentes podem ser desenvolvidos em diferentes linguagens e integrados utilizando tecnologias variadas, como Web Services, mensageria e RPC, favorecendo a interoperabilidade entre ambientes heterogêneos.

O SCA é normalmente estudado a partir de três conceitos fundamentais: composição, montagem e política. A composição corresponde à combinação de componentes e serviços para formar soluções mais amplas. A montagem define como esses elementos são conectados e configurados para operar em conjunto. Já a política estabelece requisitos de segurança e governança, incluindo autenticação, autorização e controle de acesso. Em conjunto, esses elementos permitem construir soluções corporativas estruturadas, alinhadas aos princípios da Arquitetura Orientada a Serviços.

Governança SOA

Padrões SOA

INCIDÊNCIA EM PROVA: MÉDIA

PADRÕES SOA

Os padrões SOA são soluções recorrentes para problemas comuns encontrados na construção e operação de arquiteturas orientadas a serviços. Esses padrões abrangem aspectos como contratos, composição, segurança, descoberta, integração, governança e gerenciamento, promovendo consistência e boas práticas arquiteturais.

A Arquitetura Orientada a Serviços utiliza princípios de design para reduzir a complexidade inerente à integração de múltiplos sistemas, processos e áreas organizacionais. Entre os princípios mais conhecidos estão contrato padronizado, baixo acoplamento, abstração, reusabilidade, autonomia, independência de estado, visibilidade e composição. Essas diretrizes procuram tornar os serviços mais consistentes, interoperáveis e preparados para manutenção e evolução ao longo do tempo, favorecendo a construção de soluções sustentáveis.



Embora SOA e Orientação a Objetos sejam abordagens distintas, ambas compartilham conceitos fundamentais da engenharia de software, especialmente a separação de responsabilidades. Na Orientação a Objetos, essa organização ocorre por meio de classes e objetos especializados. Já em SOA, as responsabilidades são distribuídas entre serviços que representam capacidades relevantes para o negócio. Em ambos os casos, a decomposição da complexidade busca facilitar desenvolvimento, manutenção e compreensão dos sistemas.

A principal diferença entre essas abordagens está no nível de abstração adotado. Enquanto a Orientação a Objetos concentra-se na modelagem e implementação de software por meio de classes e objetos, a SOA atua em uma escala arquitetural mais ampla, organizando capacidades e promovendo integração entre sistemas. Apesar disso, conceitos como abstração, modularidade e reutilização aparecem em

ambas as abordagens, demonstrando que compartilham fundamentos conceituais, embora tenham objetivos distintos.

A decomposição estrutural é um mecanismo utilizado por ambas as abordagens para administrar sistemas complexos. Ao dividir funcionalidades em componentes menores e especializados, torna-se possível simplificar a manutenção, facilitar a evolução tecnológica e melhorar a organização da solução. Essa estratégia reduz a complexidade operacional e permite que cada componente desempenhe responsabilidades mais bem definidas, contribuindo para a qualidade geral da arquitetura.

Entre os princípios mais valorizados em SOA destacam-se a reusabilidade, o baixo acoplamento e a autonomia. Serviços reutilizáveis podem ser empregados em diferentes soluções, reduzindo redundâncias e acelerando o desenvolvimento. O baixo acoplamento minimiza dependências entre componentes, reduzindo impactos causados por mudanças. Já a autonomia busca garantir maior controle dos serviços sobre sua lógica e recursos, favorecendo estabilidade, previsibilidade e independência operacional.

Outro princípio importante é a independência de estado. Em vez de manter informações entre múltiplas interações, os serviços procuram processar cada requisição com o mínimo possível de dependência de estados anteriores. Essa característica favorece escalabilidade, previsibilidade e melhor utilização dos recursos computacionais. Em conjunto com os demais princípios, contribui para a construção de arquiteturas distribuídas mais eficientes e preparadas para crescimento.

A orientação a serviços também enfatiza a interoperabilidade e a independência tecnológica. O objetivo não é vincular a arquitetura a uma plataforma específica, mas permitir que sistemas distintos possam interagir de forma consistente. Embora implementações concretas possam utilizar tecnologias proprietárias, os princípios de SOA procuram preservar a capacidade de integração e evolução da solução, evitando dependências excessivas de fornecedores ou produtos específicos.

Outro aspecto essencial é o alinhamento entre tecnologia e negócio. Os serviços devem representar capacidades relevantes para a organização e apoiar diretamente seus processos operacionais. Dessa forma, as decisões arquiteturais deixam de ser motivadas exclusivamente por fatores tecnológicos e passam a refletir objetivos corporativos. Esse alinhamento aumenta o valor estratégico da arquitetura e favorece sua adaptação às mudanças organizacionais e de mercado.

Thomas Erl propôs modelos de serviços que auxiliam na organização das responsabilidades dentro de uma arquitetura orientada a serviços. O primeiro deles é o Entity Service, responsável por representar entidades fundamentais do negócio, como clientes, contratos, pedidos ou produtos. Esses serviços costumam possuir elevada estabilidade e grande potencial de reutilização, pois disponibilizam capacidades amplamente utilizadas por diferentes processos e aplicações da organização.

Outro modelo é o Utility Service, voltado ao fornecimento de funcionalidades genéricas e compartilháveis. Serviços de autenticação, criptografia, auditoria, notificações e registro de eventos são exemplos típicos dessa categoria. Seu objetivo é evitar duplicação de funcionalidades técnicas em diferentes partes da arquitetura. Por serem independentes de domínios específicos de negócio, geralmente apresentam alto potencial de compartilhamento entre aplicações e serviços distintos.

O terceiro modelo é o Task Service, criado para executar tarefas ou processos específicos de negócio. Diferentemente dos Entity Services e Utility Services, seu foco não está na disponibilização de capacidades genéricas, mas na coordenação de atividades relacionadas a fluxos operacionais específicos. Embora normalmente apresentem menor potencial de reutilização, desempenham papel fundamental ao transformar capacidades reutilizáveis em processos concretos capazes de atender às necessidades da organização.

A combinação entre Entity Services, Utility Services e Task Services permite equilibrar reutilização, especialização e alinhamento com o negócio. Complementando essa estrutura, a composição de serviços possibilita integrar funcionalidades independentes para formar soluções mais amplas e sofisticadas. Dessa forma, a organização aproveita capacidades já existentes, reduz retrabalho e obtém maior flexibilidade para evoluir seus sistemas sem precisar reconstruir funcionalidades previamente disponibilizadas em seu ecossistema de serviços.

QUESTÕES COMENTADAS

1. (FGV / ALEGO - 2026) A ALEGO, em processo de transformação digital e integração de sistemas legados, decide adotar uma arquitetura SOA para melhorar interoperabilidade e agilidade. No entanto enfrenta alguns desafios ao implementar a arquitetura. Foram levantados pela equipe alguns desafios:

I. A integração de diferentes serviços, quando esses serviços são desenvolvidos em diferentes plataformas ou linguagens.

II. A comunicação entre serviços, quando são necessárias múltiplas chamadas para completar uma única operação de negócio.

III. A reutilização de serviços existentes, para permitir que diferentes aplicações acessem os mesmos serviços de negócio.

IV. Enfoque em práticas de compartilhamento, onde os serviços são projetados para serem reutilizados em várias aplicações dentro de uma organização.

Com base nos desafios levantados, está correto o que se afirma em

- a) I e II apenas.
- b) III e IV apenas.
- c) I, II e III apenas.
- d) II apenas.
- e) I, II, III e IV.

Comentários:

(I) Correto. Integrar serviços feitos em plataformas e linguagens diferentes é um desafio clássico em SOA, porque exige padronização e interoperabilidade entre tecnologias distintas.

(II) Correto. Quando uma operação de negócio depende de várias chamadas entre serviços, a comunicação fica mais complexa, com impacto em desempenho e orquestração.

(III) Errado. A reutilização de serviços existentes é mais uma vantagem e objetivo da SOA do que propriamente um desafio levantado no contexto da questão;

(IV) Errado. O compartilhamento e o reuso em várias aplicações representam um princípio da SOA, não um desafio entre os itens considerados.

Gabarito: Letra A

2. (FGV / DPE RO - 2025) A arquitetura orientada a serviços é uma abordagem de design de software que se concentra na construção de sistemas de software funcionais, modulares e escaláveis a partir de componentes individuais. Com relação às vantagens dessa arquitetura em comparação com a abordagem monolítica de desenvolvimento de software, analise os itens a seguir.

I. Manutenção simplificada - A baixa integração e reutilização de serviços faz com que seja mais seguro desenvolver novas aplicações. Os desenvolvedores precisam sempre criar serviços do zero assim como nas aplicações monolíticas.

II. Adaptabilidade - Preserva a compatibilidade de antigas versões dos serviços e facilita o planejamento serviços futuros. Os desenvolvedores podem trazer serviços mais antigos para um ambiente operacional mais novo, até mesmo substituir serviços desatualizados por versões mais recentes que aproveitam hardwares atualizados.

III. Escalabilidade - Como os serviços são autossuficientes e independentes podem ser monitorados pela camada de gerenciamento do SOA, se o desempenho de um serviço cair devido ao aumento da demanda, novas instâncias poderão ser iniciadas. As aplicações que utilizam o serviço não saberão sobre as novas instâncias, apenas perceberão que os SLA permanecem aceitáveis.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) I e II, apenas.
- d) I e III, apenas.
- e) II e III, apenas.

Comentários:

(I) Errado. O item contraria a lógica do SOA ao falar em baixa reutilização e criação de serviços sempre do zero. Justamente uma das vantagens dessa arquitetura é reaproveitar serviços e reduzir acoplamento, o que ajuda na manutenção.

(II) Correto. Aqui a ideia é a flexibilidade do SOA. A preservação de compatibilidade e a possibilidade de evoluir ou substituir serviços facilitam bastante a adaptação do sistema a novos ambientes e tecnologias.

(III) Correto. Como os serviços são independentes, dá para escalar de forma mais pontual. Se a demanda cresce, novas instâncias podem ser ativadas sem impactar quem consome o serviço, mantendo o desempenho esperado.

Gabarito: Letra E

3. (FGV / CPRM - 2025) Considerando a Arquitetura Orientada a Serviços (SOA), seus componentes e características, assinale a afirmação correta.

- a) WSDL é um formato JSON que especifica como acessar qualquer tipo de serviço.
- b) UDDI é um formato XML que inclui os dados que serão transmitidos ou recebidos de um serviço Web.
- c) Os 3 principais elementos do formato SOAP são o Cabeçalho, Corpo e Envelope, sendo exclusivamente neste último, onde estão os dados.
- d) A independência de outros sistemas da Arquitetura Orientada a Serviços é provida, em grande parte, pelo fraco (baixo) acoplamento.
- e) A Arquitetura Orientada a Serviços possui padrões proprietários definidos pelos seus respectivos provedores de serviços de forma a garantir a segurança.

Comentários:

- (a) Errado. WSDL não é JSON; ele descreve serviços Web em XML, mostrando operações, mensagens e formas de acesso.
- (b) Errado. UDDI funciona como diretório de serviços para publicação e descoberta, não como formato com os dados trocados pelo serviço.
- (c) Errado. No SOAP, o Envelope envolve a mensagem, o Header traz metadados e o Body carrega o conteúdo principal, não o Envelope.
- (d) Correto. Em SOA, o baixo acoplamento ajuda bastante na independência entre sistemas, porque os serviços se comunicam com menos dependência interna.
- (e) Errado. SOA se apoia em padrões amplamente adotados para interoperabilidade; a ideia central não é usar padrões proprietários de cada provedor.

Gabarito: Letra D

4. (FGV / ALETO - 2024) A respeito da Arquitetura Orientada a Serviços (SOA), analise as afirmativas a seguir.

I. Dentro de uma arquitetura orientada a serviços (SOA), a transferência bancária usando PIX estimula que cada instituição financeira desenvolva seu próprio protocolo de comunicação.

II. O pagamento de boletos utilizando o PIX, inserido em uma SOA, exige mecanismos de segurança robustos.

III. Um problema relevante na implementação de SOA é a gestão de serviços, que pode se tornar complexa necessitando de ferramentas eficientes de governança.

Está correto o que se afirma em

- a) II e III, apenas
- b) I e III, apenas.
- c) I e II, apenas.
- d) III, apenas.
- e) II, apenas.

Comentários:

(I) Errado. Em SOA, a ideia é justamente favorecer integração com padrões e serviços bem definidos, não incentivar cada instituição a criar um protocolo próprio, o que dificultaria a interoperabilidade.

(II) Correto. Pagamentos via PIX lidam com dados sensíveis e transações financeiras, então, dentro de uma SOA, mecanismos fortes de segurança são indispensáveis para proteger comunicação e operação.

(III) Correto. Um ponto bem importante em SOA é a gestão dos serviços. Como há vários serviços integrados, a governança pode ficar mais complexa e pedir ferramentas eficientes de controle.

Gabarito: Letra A

5. (FGV / SES MT - 2024) Observe a figura a seguir, que representa a estrutura básica do SOA e assinale (V) para a afirmativa verdadeira e (F) para falsa



() Consumidor de Serviço são as aplicações que consultam o Provedor de serviços para invocar os serviços do Registro de Serviços.

() O Provedor de Serviço implementa e disponibiliza serviços, publicando suas descrições para que possam ser descobertas e consumidas pelas aplicações.

() O Registro de Serviços armazena e disponibiliza as descrições dos serviços publicados.

() Na SOA, a separação de assuntos envolve dividir problemas grandes em partes menores, facilitando a decomposição da solução em diferentes componentes específicos.

As afirmativas são, respectivamente,

- a) F - V - V - V.
- b) V - F - V - F.
- c) F - V - F - V.
- d) V - F - F - F.

Comentários:

(F) O consumidor de serviço consulta o Registro de Serviços para localizar serviços e depois se liga ao Provedor de Serviço. Ele não invoca serviços do registro;

(V) O Provedor de Serviço implementa e disponibiliza serviços, publicando suas descrições para descoberta e consumo;

(V) O Registro de Serviços armazena e disponibiliza as descrições dos serviços publicados pelos provedores;

(V) A separação de assuntos (Separation of Concerns) promove a divisão de problemas complexos em componentes menores e especializados.

Gabarito: Letra A

6. (FGV / INPE - 2024) O Projeto Orientado a Serviços (SOA) tem como objetivo facilitar a integração entre diferentes serviços. Sobre o emprego de SOA, assinale a afirmativa correta.

- a) Permite o compartilhamento direto de bancos de dados entre diferentes serviços para reutilização de funcionalidades.
- b) Busca elevar o grau de acoplamento entre serviços com o objetivo de garantir compatibilidade.
- c) A lógica é abstraída através de interfaces de serviço, de modo a permitir a integração entre sistemas heterogêneos.
- d) A segurança é intrinsecamente garantida pela arquitetura, eliminando a necessidade de protocolos de segurança adicionais.
- e) Não é compatível com o padrão SOAP, por conta de sua restrição ao uso de XML para formatação de mensagens.

Comentários:

- (a) Errado. Em SOA, a ideia não é compartilhar banco de dados diretamente entre serviços, porque isso aumenta dependência e dificulta a autonomia de cada um.
- (b) Errado. SOA busca justamente reduzir o acoplamento, para que os serviços se comuniquem com mais independência e flexibilidade.
- (c) Correto. Em SOA, a lógica fica exposta por interfaces de serviço, o que facilita a integração entre sistemas diferentes, mesmo com tecnologias distintas.
- (d) Errado. SOA não garante segurança por si só. Na prática, ainda é preciso usar mecanismos e protocolos específicos para proteger a comunicação.
- (e) Errado. SOA é compatível com SOAP, que inclusive foi bastante usado nesse modelo de arquitetura para troca de mensagens entre serviços.

Gabarito: Letra C

7. (FGV / TJ MT - 2024) Um dos estilos de arquitetura de software mais usados nos últimos anos, dadas as suas vantagens em relação aos demais existentes, é a arquitetura orientada a serviços (Service-Oriented Architecture - SOA). A arquitetura orientada a serviços (SOA)

- a) organiza a estrutura de um sistema como um conjunto de serviços independentes que se comunicam por meio de protocolos padrão, como HTTP e SOAP.
- b) utiliza uma abordagem na qual os serviços são acoplados de forma estreita para garantir maior controle e interdependência entre as partes do sistema.
- c) é uma arquitetura em que todos os serviços são integrados em um sistema monolítico, garantindo alta coesão entre os módulos.
- d) exige que todos os serviços sejam implementados na mesma linguagem de programação para garantir consistência no sistema.
- e) utiliza a comunicação síncrona entre serviços para manter a consistência e a segurança dos dados transmitidos no sistema.

Comentários:

- (a) Correto. Na SOA, o sistema é dividido em serviços independentes, que conversam entre si por protocolos padrão, como HTTP e SOAP. Isso facilita integração, reutilização e interoperabilidade.
- (b) Errado. SOA busca justamente reduzir o acoplamento entre os serviços, deixando cada parte mais autônoma e flexível para evoluir sem depender tanto das outras.
- (c) Errado. SOA não segue a ideia de sistema monolítico. A proposta é separar funcionalidades em serviços distintos, e não concentrar tudo em um único bloco.
- (d) Errado. Um ponto forte da SOA é permitir serviços feitos em tecnologias e linguagens diferentes, desde que consigam se comunicar por padrões bem definidos.
- (e) Errado. SOA não exige comunicação apenas síncrona. Dependendo do cenário, pode haver comunicação síncrona ou assíncrona entre os serviços.

Gabarito: Letra A

8. (FGV / MPE SC - 2022) A adoção da Arquitetura Orientada a Serviços (Service Oriented Architecture - SOA) viabiliza a utilização de vários recursos que proporcionam benefícios aos softwares que suportam as operações de negócios das organizações.

A detecção de ataques de segurança, como ataques de negação de serviço, é um benefício proporcionado pelo recurso de SOA denominado:

- a) Orquestração de Serviços;
- b) Coreografia de Serviços;
- c) Monitoramento de Mensagens;
- d) Virtualização de Serviços;
- e) Processamento de Eventos Complexos.

Comentários:

(a) Errado. Orquestração organiza e coordena a execução de serviços em um fluxo, mas não tem como foco principal identificar ataques como negação de serviço.

(b) Errado. Coreografia trata da interação entre serviços de forma distribuída, definindo como eles se comunicam, e não da detecção de ameaças.

(c) Correto. Monitoramento de mensagens permite acompanhar o tráfego e o conteúdo das trocas entre serviços, ajudando a perceber padrões anômalos e possíveis ataques.

(d) Errado. Virtualização de serviços facilita testes, simulações e desacoplamento, mas não é o recurso voltado à detecção de ataques.

(e) Errado. Processamento de eventos complexos analisa eventos e correlações, mas, no contexto da questão, o recurso associado à detecção é o monitoramento de mensagens.

Gabarito: Letra C

9. (FGV / MPE AL - 2018) A Arquitetura Orientada a Serviços (SOA - Service Oriented Architecture) é uma maneira de arquitetar o software na qual as funcionalidades da aplicação são expostas na forma de serviços. Sua implementação segue os princípios de design.

De acordo com os princípios do SOA, ao delegar o controle da lógica encapsulada no serviço para uma classe estranha ao serviço, é ferido o princípio da

- a) abstração do serviço.
- b) reusabilidade do serviço.
- c) autonomia do serviço.
- d) composição do serviço.
- e) baixo acoplamento do serviço.

Comentários:

- (a) Errado. Abstração do serviço trata de esconder a lógica interna e expor só o necessário, não do controle dessa lógica por elemento externo.
- (b) Errado. Reusabilidade tem a ver com criar serviços aproveitáveis em vários contextos, não com a delegação de controle para outra classe.
- (c) Correto. Autonomia exige que o serviço controle sua própria lógica e comportamento. Se esse controle vai para uma classe estranha ao serviço, esse princípio é quebrado.
- (d) Errado. Composição diz respeito à combinação de serviços para formar processos maiores, não ao controle interno da lógica encapsulada.
- (e) Errado. Baixo acoplamento busca reduzir dependências entre serviços, mas o ponto central aqui é a perda de controle interno, ligada à autonomia.

Gabarito: Letra C**10.(FGV / IBGE - 2016) SOA é um estilo arquitetônico de software usado para construir soluções empresariais baseadas em serviços Web. São características dos serviços desse estilo:**

- a) encapsulamento e recursividade;
- b) modularidade e transparência;
- c) reuso e descoberta dinâmica;
- d) autonomia e dependência de protocolos Web;
- e) isolamento de responsabilidades e paralelismo de dados.

Comentários:

- (a) Errado. Encapsulamento é uma ideia comum em software, mas recursividade não é traço típico usado para caracterizar serviços em SOA.
- (b) Errado. Modularidade ajuda a entender SOA, mas transparência não aparece como característica central dos serviços nesse estilo.
- (c) Correto. Em SOA, os serviços são pensados para reuso e podem ser localizados por mecanismos de descoberta dinâmica, o que dá mais flexibilidade.

(d) Errado. Autonomia combina com SOA, mas dependência de protocolos Web contraria a ideia de baixo acoplamento e interoperabilidade.

(e) Errado. Isolamento de responsabilidades até se aproxima do tema, mas paralelismo de dados não é característica típica dos serviços em SOA.

Gabarito: Letra C

11.(FGV / Paulínia - 2016) Para atender seu contexto de negócio, a empresa “Solução & Associados” possui diversas aplicações desenvolvidas em diferentes plataformas e linguagens de programação que precisam se comunicar. Desta forma, a “Solução & Associados” resolveu empregar a Arquitetura Orientada a Serviços (SOA) com a utilização de um barramento de serviços, ou Enterprise Service Bus (ESB).

Assim, a utilização desse barramento

- a) representa uma infraestrutura baseada em Web Services.
- b) proporciona maior acoplamento entre consumidores e provedores de serviço.
- c) facilita a criação de novos serviços construídos a partir de agregados de outros serviços.
- d) regulamenta a forma como os serviços se comunicam e interagem uns com os outros.
- e) é considerada o dono do serviço que está sendo disponibilizado.

Comentários:

(a) Errado. O ESB pode até integrar Web Services, mas não se resume a uma infraestrutura baseada neles; ele vai além, conectando serviços e aplicações de naturezas diferentes.

(b) Errado. A ideia do ESB é justamente reduzir dependências diretas, deixando a comunicação mais desacoplada entre quem consome e quem fornece o serviço.

(c) Errado. Embora possa apoiar composições, esse não é o ponto central da alternativa. O foco do barramento está em mediar e organizar a comunicação entre serviços.

(d) Correto. O ESB atua como um barramento que padroniza e organiza como os serviços trocam mensagens e interagem, facilitando a integração entre sistemas distintos.

(e) Errado. O ESB não é “dono” do serviço. Ele funciona como intermediário na comunicação, sem assumir a propriedade do serviço disponibilizado.

Gabarito: Letra D

12.(FGV / CODEBA - 2016) O Manifesto SOA, divulgado durante o segundo Simpósio Internacional de Arquitetura Orientada a Serviços (SOA), em 2009, definiu o conjunto de itens, objetivos e princípios orientadores que visam proporcionar um entendimento claro de SOA e Orientação a Serviço.

De acordo com o manifesto, a Orientação a Serviço é um paradigma que molda o que você faz. Já Arquitetura Orientada a Serviço (SOA) é um tipo de arquitetura que resulta da aplicação de orientação a serviço.

Sobre os itens e objetivos definidos pelo Manifesto SOA, assinale a afirmativa correta.

- a) A estratégia técnica dever ser priorizada em relação ao valor de negócio.
- b) Os benefícios específicos de projetos devem ser priorizados em relação aos objetivos estratégicos.
- c) As implementações de propósito específico devem ser priorizadas em relação aos serviços compartilhados.
- d) A flexibilidade deve ser priorizada em relação à otimização.
- e) A perfeição inicial deve ser priorizada em relação ao refinamento evolutivo.

Comentários:

- (a) Errado. No Manifesto SOA, o foco não é colocar a estratégia técnica acima do valor de negócio; a ideia é que a tecnologia sirva ao negócio.
- (b) Errado. O manifesto valoriza objetivos estratégicos mais amplos, e não ganhos isolados de projetos específicos.
- (c) Errado. A proposta de SOA privilegia reaproveitamento e compartilhamento, não implementações fechadas para um único propósito.
- (d) Correto. O manifesto destaca a flexibilidade como prioridade, porque ela permite adaptação e evolução dos serviços ao longo do tempo.
- (e) Errado. Em SOA, o refinamento evolutivo é mais valorizado do que buscar uma perfeição inicial logo de saída.

Gabarito: Letra D

LISTA DE QUESTÕES

1. (FGV / ALEGO - 2026) A ALEGO, em processo de transformação digital e integração de sistemas legados, decide adotar uma arquitetura SOA para melhorar interoperabilidade e agilidade. No entanto enfrenta alguns desafios ao implementar a arquitetura. Foram levantados pela equipe alguns desafios:

I. A integração de diferentes serviços, quando esses serviços são desenvolvidos em diferentes plataformas ou linguagens.

II. A comunicação entre serviços, quando são necessárias múltiplas chamadas para completar uma única operação de negócio.

III. A reutilização de serviços existentes, para permitir que diferentes aplicações acessem os mesmos serviços de negócio.

IV. Enfoque em práticas de compartilhamento, onde os serviços são projetados para serem reutilizados em várias aplicações dentro de uma organização.

Com base nos desafios levantados, está correto o que se afirma em

- a) I e II apenas.
- b) III e IV apenas.
- c) I, II e III apenas.
- d) II apenas.
- e) I, II, III e IV.

2. (FGV / DPE RO - 2025) A arquitetura orientada a serviços é uma abordagem de design de software que se concentra na construção de sistemas de software funcionais, modulares e escaláveis a partir de componentes individuais. Com relação às vantagens dessa arquitetura em comparação com a abordagem monolítica de desenvolvimento de software, analise os itens a seguir.

I. Manutenção simplificada - A baixa integração e reutilização de serviços faz com que seja mais seguro desenvolver novas aplicações. Os desenvolvedores precisam sempre criar serviços do zero assim como nas aplicações monolíticas.

II. Adaptabilidade - Preserva a compatibilidade de antigas versões dos serviços e facilita o planejamento serviços futuros. Os desenvolvedores podem trazer serviços mais antigos para

um ambiente operacional mais novo, até mesmo substituir serviços desatualizados por versões mais recentes que aproveitam hardwares atualizados.

III. Escalabilidade - Como os serviços são autossuficientes e independentes podem ser monitorados pela camada de gerenciamento do SOA, se o desempenho de um serviço cair devido ao aumento da demanda, novas instâncias poderão ser iniciadas. As aplicações que utilizam o serviço não saberão sobre as novas instâncias, apenas perceberão que os SLA permanecem aceitáveis.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) I e II, apenas.
- d) I e III, apenas.
- e) II e III, apenas.

3. (FGV / CPRM - 2025) Considerando a Arquitetura Orientada a Serviços (SOA), seus componentes e características, assinale a afirmação correta.

- a) WSDL é um formato JSON que especifica como acessar qualquer tipo de serviço.
- b) UDDI é um formato XML que inclui os dados que serão transmitidos ou recebidos de um serviço Web.
- c) Os 3 principais elementos do formato SOAP são o Cabeçalho, Corpo e Envelope, sendo exclusivamente neste último, onde estão os dados.
- d) A independência de outros sistemas da Arquitetura Orientada a Serviços é provida, em grande parte, pelo fraco (baixo) acoplamento.
- e) A Arquitetura Orientada a Serviços possui padrões proprietários definidos pelos seus respectivos provedores de serviços de forma a garantir a segurança.

4. (FGV / ALETO - 2024) A respeito da Arquitetura Orientada a Serviços (SOA), analise as afirmativas a seguir.

I. Dentro de uma arquitetura orientada a serviços (SOA), a transferência bancária usando PIX estimula que cada instituição financeira desenvolva seu próprio protocolo de comunicação.

II. O pagamento de boletos utilizando o PIX, inserido em uma SOA, exige mecanismos de segurança robustos.

III. Um problema relevante na implementação de SOA é a gestão de serviços, que pode se tornar complexa necessitando de ferramentas eficientes de governança.

Está correto o que se afirma em

- a) II e III, apenas
- b) I e III, apenas.
- c) I e II, apenas.
- d) III, apenas.
- e) II, apenas.

5. (FGV / SES MT - 2024) Observe a figura a seguir, que representa a estrutura básica do SOA e assinale (V) para a afirmativa verdadeira e (F) para falsa



() Consumidor de Serviço são as aplicações que consultam o Provedor de serviços para invocar os serviços do Registro de Serviços.

() O Provedor de Serviço implementa e disponibiliza serviços, publicando suas descrições para que possam ser descobertas e consumidas pelas aplicações.

() O Registro de Serviços armazena e disponibiliza as descrições dos serviços publicados.

() Na SOA, a separação de assuntos envolve dividir problemas grandes em partes menores, facilitando a decomposição da solução em diferentes componentes específicos.

As afirmativas são, respectivamente,

- a) F - V - V - V.
- b) V - F - V - F.
- c) F - V - F - V.
- d) V - F - F - F.

6. (FGV / INPE - 2024) O Projeto Orientado a Serviços (SOA) tem como objetivo facilitar a integração entre diferentes serviços. Sobre o emprego de SOA, assinale a afirmativa correta.

- a) Permite o compartilhamento direto de bancos de dados entre diferentes serviços para reutilização de funcionalidades.
- b) Busca elevar o grau de acoplamento entre serviços com o objetivo de garantir compatibilidade.
- c) A lógica é abstraída através de interfaces de serviço, de modo a permitir a integração entre sistemas heterogêneos.
- d) A segurança é intrinsecamente garantida pela arquitetura, eliminando a necessidade de protocolos de segurança adicionais.
- e) Não é compatível com o padrão SOAP, por conta de sua restrição ao uso de XML para formatação de mensagens.

7. (FGV / TJ MT - 2024) Um dos estilos de arquitetura de software mais usados nos últimos anos, dadas as suas vantagens em relação aos demais existentes, é a arquitetura orientada a serviços (Service-Oriented Architecture - SOA). A arquitetura orientada a serviços (SOA)

- a) organiza a estrutura de um sistema como um conjunto de serviços independentes que se comunicam por meio de protocolos padrão, como HTTP e SOAP.
- b) utiliza uma abordagem na qual os serviços são acoplados de forma estreita para garantir maior controle e interdependência entre as partes do sistema.
- c) é uma arquitetura em que todos os serviços são integrados em um sistema monolítico, garantindo alta coesão entre os módulos.
- d) exige que todos os serviços sejam implementados na mesma linguagem de programação para garantir consistência no sistema.
- e) utiliza a comunicação síncrona entre serviços para manter a consistência e a segurança dos dados transmitidos no sistema.

8. (FGV / MPE SC - 2022) A adoção da Arquitetura Orientada a Serviços (Service Oriented Architecture - SOA) viabiliza a utilização de vários recursos que proporcionam benefícios aos softwares que suportam as operações de negócios das organizações.

A detecção de ataques de segurança, como ataques de negação de serviço, é um benefício proporcionado pelo recurso de SOA denominado:

- a) Orquestração de Serviços;
- b) Coreografia de Serviços;
- c) Monitoramento de Mensagens;
- d) Virtualização de Serviços;
- e) Processamento de Eventos Complexos.

9. (FGV / MPE AL - 2018) A Arquitetura Orientada a Serviços (SOA - Service Oriented Architecture) é uma maneira de arquitetar o software na qual as funcionalidades da aplicação são expostas na forma de serviços. Sua implementação segue os princípios de design.

De acordo com os princípios do SOA, ao delegar o controle da lógica encapsulada no serviço para uma classe estranha ao serviço, é ferido o princípio da

- a) abstração do serviço.
- b) reusabilidade do serviço.
- c) autonomia do serviço.
- d) composição do serviço.
- e) baixo acoplamento do serviço.

10.(FGV / IBGE - 2016) SOA é um estilo arquitetônico de software usado para construir soluções empresariais baseadas em serviços Web. São características dos serviços desse estilo:

- a) encapsulamento e recursividade;
- b) modularidade e transparência;
- c) reuso e descoberta dinâmica;
- d) autonomia e dependência de protocolos Web;
- e) isolamento de responsabilidades e paralelismo de dados.

11.(FGV / Paulínia - 2016) Para atender seu contexto de negócio, a empresa "Solução & Associados" possui diversas aplicações desenvolvidas em diferentes plataformas e linguagens de programação que precisam se comunicar. Desta forma, a "Solução & Associados" resolveu empregar a Arquitetura Orientada a Serviços (SOA) com a utilização de um barramento de serviços, ou Enterprise Service Bus (ESB).

Assim, a utilização desse barramento

- a) representa uma infraestrutura baseada em Web Services.
- b) proporciona maior acoplamento entre consumidores e provedores de serviço.
- c) facilita a criação de novos serviços construídos a partir de agregados de outros serviços.

- d) regulamenta a forma como os serviços se comunicam e interagem uns com os outros.
- e) é considerada o dono do serviço que está sendo disponibilizado.

12.(FGV / CODEBA - 2016) O Manifesto SOA, divulgado durante o segundo Simpósio Internacional de Arquitetura Orientada a Serviços (SOA), em 2009, definiu o conjunto de itens, objetivos e princípios orientadores que visam proporcionar um entendimento claro de SOA e Orientação a Serviço.

De acordo com o manifesto, a Orientação a Serviço é um paradigma que molda o que você faz. Já Arquitetura Orientada a Serviço (SOA) é um tipo de arquitetura que resulta da aplicação de orientação a serviço.

Sobre os itens e objetivos definidos pelo Manifesto SOA, assinale a afirmativa correta.

- a) A estratégia técnica deve ser priorizada em relação ao valor de negócio.
- b) Os benefícios específicos de projetos devem ser priorizados em relação aos objetivos estratégicos.
- c) As implementações de propósito específico devem ser priorizadas em relação aos serviços compartilhados.
- d) A flexibilidade deve ser priorizada em relação à otimização.
- e) A perfeição inicial deve ser priorizada em relação ao refinamento evolutivo.

13. Gabarito

- | | | | | | |
|----|---------|----|---------|-----|---------|
| 1. | LETRA A | 5. | LETRA A | 9. | LETRA C |
| 2. | LETRA E | 6. | LETRA C | 10. | LETRA C |
| 3. | LETRA D | 7. | LETRA A | 11. | LETRA D |
| 4. | LETRA A | 8. | LETRA C | 12. | LETRA D |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.